

Programação Orientada a Objetos

Capítulo 1 — Introdução à Orientação a Objetos

Aula 01

O que veremos hoje?

- 1 Conceitos introdutórios da Orientação a Objetos
- 2 Definição de Orientação a Objetos
- 3 Classes: superclasses, subclasses, abstratas e concretas
- 4 Objetos: instâncias, atributos e métodos
- 5 Mensagens e troca de informações entre objetos
- 6 Dinâmicas & Atividades práticas**



1.1 Conceitos Introdutórios

Origens e evolução do paradigma de OO

O começo: décadas de 1950–1960

- Nas décadas de 1950 e 1960, operar e programar computadores exigia formação especializada em computação;
- Programação inicial baseava-se em código binário e endereçamento direto de memória;
- Com a evolução, surgiram linguagens de alto nível, mais próximas da comunicação humana;
- Computadores ficaram menores, mais baratos e acessíveis a públicos diversificados;
- Os paradigmas de programação passaram a buscar maior semelhança com o mundo real.

Por que surge a Orientação a Objetos?

- Necessidade corporativa: programação deve atender processos de negócio reais;
- Objetivo principal: modelar problemas usando abstração de objetos do mundo real;
- Identificar características (atributos) e funcionalidades (métodos) reais de cada entidade;
- Facilitar a compreensão dos programadores sobre o processo de negócio, não apenas requisitos técnicos;
- Resultado: metodologias mais rápidas, dinâmicas e compreensíveis.

Dinâmica 1 — Antes vs. Depois da OO

- Em duplas, discutam o cenário abaixo (5 minutos):
 - Imagine que vocês são programadores nos anos 1960. O chefe pede um sistema para controlar estoque de uma loja;
 - Como seria modelar esse problema em código binário/sequencial?
- Agora, pensem no mesmo problema com orientação a objetos: quais 'objetos' da loja vocês usariam?
 - Listem pelo menos 3 objetos e 2 características de cada um;
 - Compartilhem as respostas com a turma!

Respostas possíveis — Dinâmica 1

- Objetos típicos:
 - Produto (nome, preço, quantidade);
 - Cliente (nome, CPF);
 - Venda (data, valor total);
 - Funcionário (nome, cargo);
- No modelo sequencial: tudo seria tratado como variáveis soltas, sem agrupamento lógico;
- Na OO: cada entidade é uma classe com seus atributos organizados — muito mais próximo da realidade da loja;
- Vantagem visível: ao explicar o sistema para o dono da loja, os termos são os mesmos do cotidiano.



1.2 Definição de OO

Paradigma baseado na troca de mensagens entre objetos

O que é Orientação a Objetos?

- Paradigma de análise, projeto e programação de softwares baseado em troca de mensagens entre objetos;
- Ideia central: abstrair computacionalmente os elementos relevantes do cenário real;
- Cada elemento possui características principais (atributos) e ações pertinentes (métodos);
- Exemplo: numa oficina mecânica, temos clientes, carros, ferramentas, mecânicos — todos representáveis como classes;
- Cada carro é um 'objeto' diferente, mantendo semelhança forte com a realidade observada.

Vantagens do paradigma OO

- Desenvolvimento mais ágil, seguro e preciso;
- Facilita o diálogo entre analista e usuário final — termos do cotidiano, sem jargões de TI;
- Programadores podem se especializar em classes específicas, sem necessidade de ver o sistema inteiro;
- Maior segurança: não é necessário expor todas as regras de negócio a todos os programadores;
- Programador se aproxima do negócio da empresa e compreende melhor o fluxo real da informação.

Dinâmica 2 — Mapeando uma Empresa Real

- Em grupos de 3–4, escolham uma empresa fictícia (ex.: hospital, escola, restaurante);
- Identifiquem pelo menos 5 'objetos' presentes no dia a dia dessa empresa;
- Para cada objeto, definam: 2 características (atributos) e 1 ação (método);
- Exemplo: Hospital → Médico (nome, CRM) → atenderPaciente();
- Tempo: 7 minutos. Depois, cada grupo apresenta em 2 minutos.

Exemplos de respostas — Dinâmica 2

- RESTAURANTE:
 - Garçom (nome, seção) → anotarPedido();
 - Prato (nome, preço) → calcularTempo();
 - Mesa (número, capacidade) → reservar().
- ESCOLA:
 - Aluno (matrícula, nome) → realizarProva();
 - Professor (disciplina, salário) → ministrarAula();
 - Turma (código, horário) → fecharNotas().
- O importante é notar como cada entidade tem identidade, características e comportamentos próprios.



1.3 Classes

A estrutura que organiza e agrupa objetos similares

O que é uma Classe?

- Estrutura que organiza e agrupa objetos com características parecidas e funções similares;
- Funciona como um molde: todos os objetos instanciados seguem o mesmo padrão;
- Cada objeto tem suas particularidades — valores diferentes nos mesmos atributos;
- Exemplo: Classe 'Carro' com propriedades ano, modelo e cor. Cada carro tem valores diferentes;
- Classe 'Pessoa' teria propriedades completamente diferentes: motorista, nascimento, nacionalidade.

Superclasses e Subclasses

- Superclasses podem originar outras classes, passando propriedades e métodos — conceito de herança;
- As novas classes geradas são chamadas subclasses e herdam tudo da 'classe pai';
- Subclasses podem ter alterações e propriedades adicionais em relação à superclasse;
- Exemplo: Superclasse 'Carro' → Subclasse 'Pick-up' herda ano, modelo, cor e adiciona 'caçamba';
- Herança permite reutilização de código e organização hierárquica do sistema.

Classes Abstratas vs. Concretas

- Classes Concretas: classes tradicionais das quais instanciamos objetos diretamente;
- Classes Abstratas: não podemos instanciar objetos a partir delas diretamente;
- Servem como estrutura de referência para derivar outras classes (concretas ou abstratas);
- Possibilitam herança de características por novas classes que então instanciam objetos;
- Resumo: classes abstratas são superclasses que nunca terão objetos instanciados.

Dinâmica 3 — Criando uma Hierarquia de Classes

- Em duplas, criem uma hierarquia de classes para um e-commerce:
 - Definam 1 superclasse abstrata com pelo menos 3 atributos;
 - Criem 2 subclasses concretas que herdem da superclasse e adicionem atributos próprios;
 - Exemplo de partida: Superclasse 'Produto' → Subclasses: ? (...);
 - Desenhem a hierarquia no papel e apresentem em 2 minutos.
- Tempo: 6 minutos.

Exemplo de resposta — Dinâmica 3

- **Superclasse abstrata 'Produto'**: nome (caractere), preço (real), peso (real);
- **Subclasse 'Eletrônico'**: herda tudo de Produto + voltagem (inteiro) + garantiaMeses (inteiro);
- **Subclasse 'Vestuário'**: herda tudo de Produto + tamanho (caractere) + cor (caractere);
- 'Produto' é abstrata porque não instanciamos um 'produto genérico' — sempre é algo específico.



1.4 Objetos

Instâncias de classes com atributos e métodos

Atributos e Métodos de um Objeto

- Objetos são o resultado da instanciação de uma classe — são o 'produto feito no molde';
- **Atributos:** propriedades que definem o estado do objeto (ex.: ano=2015, cor='Amarelo');
- **Métodos:** ações que o objeto pode realizar (ex.: ligar(), acelerar(), frear());
- **Método Construtor:** inicializa os atributos e aloca espaço em memória. Tem o mesmo nome da classe;
- **Método Destrutor:** apaga o objeto da memória quando não é mais necessário. Linguagens modernas usam *garbage collector*.

Dinâmica 4 — Instanciando Objetos

- Individualmente, usando a classe 'Carro' (ano, modelo, cor, ligar(), acelerar(), frear()):
- Instancie 2 objetos com valores diferentes para cada atributo;
- Descreva uma sequência de 3 chamadas de métodos para cada objeto;
- Exemplo: `meuCarro.ligar() → meuCarro.acelerar() → meuCarro.frear()`;
- Pergunta-bônus: o que o método construtor faz ao criar o objeto 'meuCarro'?
- Tempo: 4 minutos.

Respostas possíveis — Dinâmica 4

- Objeto 1 — `fusca: ano=1972, modelo='Fusca', cor='Azul'`. `fusca.ligar()` → `fusca.acelerar()` → `fusca.frear()`;
- Objeto 2 — `civic: ano=2024, modelo='Civic', cor='Preto'`. `civic.ligar()` → `civic.acelerar()` → `civic.frear()`;
- Método construtor `Carro()`: aloca memória e define valores iniciais dos atributos `ano`, `modelo` e `cor`;
- Ambos compartilham a mesma estrutura (classe), mas têm estados (valores) diferentes.



1.5 Mensagens

Comunicação e troca de dados entre objetos

Como objetos se comunicam?

- Mensagens são requisições enviadas de um objeto a outro para executar ações;
- A operação realizada depende dos métodos e atributos da classe do objeto receptor;
- Mensagens podem ser simples solicitações ou conter parâmetros e valores;
- Métodos do receptor podem retornar informações ao objeto solicitante ou apenas executar operações;
- A troca de mensagens é o motor do paradigma OO — processos simulam interações reais.

"O principal objetivo do paradigma de OO é modelar problemas com a abstração de objetos do mundo real."

Pontos-chave da aula

- OO modela o mundo real com classes, objetos, atributos e métodos;
- Classes são moldes; objetos são instâncias com valores específicos;
- Herança (superclasses/subclasses) permite reutilização e organização;
- Classes abstratas servem de referência e nunca são instanciadas;
- A comunicação entre objetos acontece por troca de mensagens.